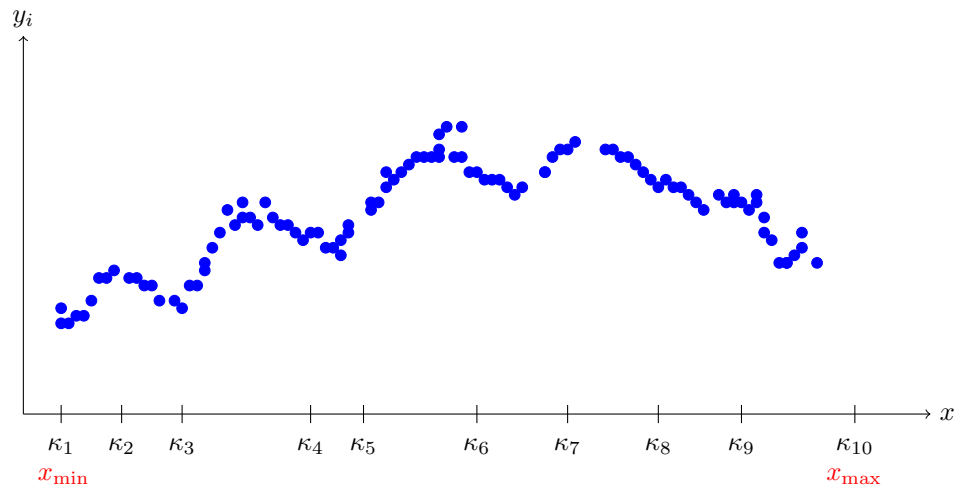
GAM

Benjamin Roelants du Vivier

April 2026

1 Splines cubiques

Le domaine de x est partionné en sous-intervalles délimités par des noeuds κ_1, κ_2, ... κ_K.



$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^{K} \gamma_k \left(x - \kappa_k\right)_+^3 \tag{1}$$

où

$$(x - \kappa_k)_+^3 = \begin{cases} (x - \kappa_k)^3 & x > \kappa_k \\ 0 & \text{sinon} \end{cases} \tag{2}$$

Intuitivement, le terme polynomial global (jusqu'à x^3) donne une tendance générale et les termes $(x - \kappa_k)_+^3$ ajoutent de la flexibilité après chaque noeud.

L'approximation (1) de la fonction f(x) est un fonction polynômiale par partie: le domaine de x est partionné en sous intervalles délimités par les noeuds κ_k et l'approximation consiste en une polynôme de degré 3 différent dans chaque sous intervalle. La fonction résultant, appellée spline cubique est continue et possède des dérivées premières et secondes continues à chaque noeud. On peut

donc résumer intuitivement les splines comme des polynômes par parties qui sont connectés à chaque noeud avec des contraintes qui assurent que cela donne une courbe unique suffisamment lisse.

L'astuce derrière les splines consiste donc, grâce au développement de Taylor à approximer le comportement inconnu f(x) par des termes connus qui sont des transformations de x: x, x^2, x^3 et $(x-\kappa_k)_+^3$, k=1,...,K. Ainsi, on retombe en fait sur un score GLM avec les variables x, x^2, x^3 et $(x-\kappa_k)_+^3$, k=1,...,K. L'algorithme IRLS permet alors de déterminer les paramètres inconnus β_0, β_1, β_2, β_3, γ_1, γ_2 ... γ_K apparaissant dans (1) en résolvant les équations de vraissemblance .

Pourquoi le degré 3 est privilégié ? Une spline de degré 3 (cubique) a des propriétés très utiles :

- Elle est continue jusqu'à la dérivée seconde ce qui implique que la courbe est lisse et sans cassures visibles.
- Elle permet de modéliser des relations non linéaires complexes sans être trop instable.
- Elle reste numériquement stable (important pour l'estimation dans les GAM).

Les splines de degrés strictement inférieurs à 3 conduisent à des modèles souvent trop rigides et visuellement peu réalistes. Quant aux splines de degré strictement supérieurs à 3, elles offrent certes plus de flexbilité mais présentent certains incovénients: oscillations parasites (phénomène de sur-ajustement), instabilité numérique accrue, interprétation plus difficile et coût de calcul plus élevé. Le degré 3 semble donc être un bon compromis ce qui explique qu'elles sont utilsiées en pratique et qu'elles suffisent pour capturer la majorité des formes observées dans les données réelles. Autre argument pour ne pas aller au delà du degré 3, on préfère contrôler la flexibilité avec le nombre de noeuds (ou une pénalisation comme on le verra plus tard) plutôt que d'augmenter le degré.

L'avantage des splines cubiques c'est que c'est simple, rapide et interprétable. Mais elles présentent également un certain nombre d'incovénients qui expliquent pourquoi d'autres types de splines leur sont préférées. Les incovénients sont les suivants:

- Sensible au nombre de noeuds choisis: risque de sous-ajustement si trop peu de noeuds (la spline est trop peu ondulée et manque une partie de l'information contenue dans les données) et risque de sur-ajustement si trop de noeuds (la spline devient trop ondulée et capte le bruit plutôt que la tendance réelle)
- Sensible au choix de la position des noeuds.

- Effets de bords: aux extérmités des données, la spline peut se comporter de façon instable. Il en résulte que les extrapolation en dehors de la plage de des données sont très peu fiables. On pourrait résumer cela en disant que: "les splines sont bonnes dans l'intervalle des données mais mauvaises en dehors". Pour rappel, le terme polynomial global (jusqu'à x^3) donne une tendance générale et les termes $(x-\kappa_k)_+^3$ ajoutent de la flexibilité après chaque noeud mais rien ne contrôle le comportement aux extrémités.

L'ensemble des fonctions génératrices est de dimension K+4 et s'écrit:

$$Base = \{1, x, x^2, x^3, (x-\kappa_1)_+^3, ..., (x-\kappa_K)_+^3\} \text{ où } k \in \{1, 2, ..., K\}$$

2 Splines cubiques naturelles

Pour pallier au défaut selon lequel les splines peuvent exploser aux bords, on impose une contrainte supplémentaire qui est que la fonction devient linéaire aux extrémités. En terme mathématiques cela signifie qu'on impose

$$f''(x) = 0 \text{ pour } x < \kappa_1 \text{ et } x > \kappa_K.$$

On part de la spline cubique:

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^{K} \gamma_k (x-\kappa_k)_+^3 \tag{3}$$

$$f''(x) = 2\beta_2 + 6\beta_3 x + 6\sum_{k=1}^{K} \gamma_k (x-\kappa_k)_+ \tag{4}$$

Pour $x > \kappa_K$

$$(x-\kappa_k)_+ = (x-\kappa_k) \quad \forall k \tag{5}$$

ce qui permet de réécrire 4 comme

$$f''(x) = 2\beta_2 + 6\beta_3 x + 6\sum_{k=1}^{K} \gamma_k (x-\kappa_k) \tag{6}$$

En développant, 6, on obtient

$$f''(x) = 2\beta_2 + 6\beta_3 x + 6x\sum_{k=1}^{K} \gamma_k - 6\sum_{k=1}^{K} \gamma_k \kappa_k$$

$$\Leftrightarrow f''(x) = \underbrace{\left[2\beta_2 - 6\sum_{k=1}^{K} \gamma_k \kappa_k\right]}_{\text{constante}} + \underbrace{\left[6\beta_3 + 6\sum_{k=1}^{K} \gamma_k\right]}_{\text{coeff de x}} \cdot x \tag{7}$$

La condition 2 impose à 7 que

$$\begin{cases} 2\beta_2 - 6\sum\limits_{k=1}^{K}\gamma_k\kappa_k = 0 \\ 6\beta_3 + 6\sum\limits_{k=1}^{K}\gamma_k = 0 \end{cases} \Leftrightarrow \begin{cases} \sum\limits_{k=1}^{K}\gamma_k\kappa_k = \dfrac{\beta_2}{3} \\ \sum\limits_{k=1}^{K}\gamma_k = -\beta_3 \end{cases} \tag{8}$$

Ces deux équations montrent que β_2 et β_3 sont entièrement déterminés par les γ_k. Par conséquent, les termes x^2 et x^3 n'apportent plus de liberté indépendante puisqu'ils sont entièrement reconstruits à partir des γ_k.
Trois options équivalentes sont alors possibles:

- *Option A: forme complète + contraintes liant β_2 et β_3 aux γ_k*

  On garde l'ensemble des termes de 3, à savoir β_0, $\beta_1 x$, $\beta_2 x^2$, $\beta_3 x^3$ et $\gamma_k\,(x-\kappa_k)_+^3$ où k=1,...K mais on impose les contraintes 8. L'interprétation est que les termes x^2 et x^3 sont bien là mais leur effet est compensé par les γ_k pour annuler la courbure aux extrémités, ce qui implique que x^2 et x^3 ne sont pas indépendants.

  Sous cette forme une spline cubique naturelle peut donc s'écrire

  $$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^{K}\gamma_k\,(x-\kappa_k)_+^3$$

  avec contraintes

  $$\sum_{k=1}^{K}\gamma_k\kappa_k = \frac{\beta_2}{3} \text{ et } \sum_{k=1}^{K}\gamma_k = -\beta_3$$

  L'ensemble des fonctions génératrices s'écrit donc

  $$\{1, x, x^2, x^3, (x-\kappa_1)_+^3, ..., (x-\kappa_K)_+^3\}$$

  S'il n'y avait aucune contrainte sur les coefficients β_0, β_1, β_2, β_3, γ_1,.. γ_K, c'est-à-dire qu'ils peuvent être choisis librementalors, oui, ces fonctions sont linéairement indépendantes et forment donc une partie libre (et donc une base). Mais en raison des 2 contraintes, β_2 et β_3 ne sont plus libres et dépendant des γ_k. La partie n'est donc pas libre car avec les contraintes, certaines fonctions peuvent être reproduites par combinaison des autres. Il en résulte que ce n'est pas une base.

  Il y a K+4 fonctions et 2 contraintes, donc le nombre de dégrés de liberté est K+2.

- *Option B: forme semi-réduite + contraintes*

  On peut montrer que sous la contraintes naturelles,

  $$\sum_{k=1}^{K}\gamma_k\kappa_k = \frac{\beta_2}{3} \text{ et } \sum_{k=1}^{K}\gamma_k = -\beta_3,$$

l'expression

$$\beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^{K} \gamma_k \left(x - \kappa_k\right)_+^3$$

peut être réécrite comme

$$\sum_{k=1}^{K} \tilde{\gamma}_k \left(x - \kappa_k\right)_+^3$$

avec contraintes

$$\sum_{k=1}^{K} \tilde{\gamma}_k \kappa_k = 0 \text{ et } \sum_{k=1}^{K} \tilde{\gamma}_k = 0$$

L'inteprétation est que les contributions des fonctions $(x - \kappa_k)_+^3$ doivent s'annuler globalement et s'annuler en tendance linéaire pour empêcher toute courbure aux extrémités.

Sous cette forme, une spline cubique naturelle peut s'écrire

$$f(x) = \beta_0 + \beta_1 x + \sum_{k=1}^{K} \tilde{\gamma}_k \left(x - \kappa_k\right)_+^3 \tag{9}$$

avec contraintes

$$\sum_{k=1}^{K} \tilde{\gamma}_k \kappa_k = 0 \text{ et } \sum_{k=1}^{K} \tilde{\gamma}_k = 0 \tag{10}$$

L'ensemble des fonctions génératrices s'écrit donc

$$\{1, x, (x - \kappa_1)_+^3, ..., (x - \kappa_K)_+^3\}$$

S'il n'y avait aucune contrainte sur les coefficients β_0, β_1, β_2, β_3, $\tilde{\gamma}_1$,.. $\tilde{\gamma}_K$, c'est-à-dire qu'ils peuvent être choisis librement alors, oui, ces fonctions sont linéairement indépendantes et forment donc une partie libre (et donc une base). Mais en raison des 2 contraintes, $\tilde{\gamma}_{K-1}$ et $\tilde{\gamma}_K$ ne sont plus libres et dépendant des $\tilde{\gamma}_k$ (avec k=1,...,K-2. La partie n'est donc pas libre car avec les contraintes, certaines fonctions peuvent être reproduites par combinaison des autres. Il en résulte que ce n'est pas une base.

Il y a K+2 paramètres et 2 contraintes, donc le nombre de degrés de liberté est K.

- *Option C: forme base indépendante*

L'idée de cette troisième option est d'intégrer les contraintes dans l'expression de la spline.

Démarrons de 9 et 10. L'idée clé est d'exprimer tous les γ_k en fonction de seulement K_2 paramètres libres. choisonns γ_1, γ_2,..., γ_{K-2} libres et exprimons γ_{K-1} et γ_K en fonction de ceux-ci en utilisant les contraintes. Cela donne

$$\begin{cases} \tilde{\gamma}_{K-1} + \tilde{\gamma}_K = -\sum_{k=1}^{K-2} \tilde{\gamma}_k \\ \tilde{\gamma}_{K-1}\kappa_{K-1} + \tilde{\gamma}_K\kappa_K = -\sum_{k=1}^{K-2} \tilde{\gamma}_k\kappa_k \end{cases} \tag{11}$$

Si on résout ce système de 2 équations à 2 inconnues, on obtient

$$\begin{cases} \tilde{\gamma}_{K-1} = -\dfrac{\kappa_K - \kappa_k}{\kappa_K - \kappa_{K-1}} \\ \tilde{\gamma}_K = -\dfrac{\kappa_{K-1} - \kappa_k}{\kappa_K - \kappa_{K-1}} \end{cases} . \tag{12}$$

En sortant les 2 derniers termes de la somme dans 10, on obtient:

$$f(x) = \beta_0 + \beta_1 x + \sum_{k=1}^{K-2} \tilde{\gamma}_k \, (x - \kappa_k)_+^3 + \tilde{\gamma}_{K-1} \, (x - \kappa_{K-1})_+^3 + \tilde{\gamma}_K \, (x - \kappa_K)_+^3 \tag{13}$$

En injectant 11 dans 13, on obtient finalement que sous cette forme, une spline cubique naturelle peut s'écrire

$$f(x) = \beta_0 + \beta_1 x + \sum_{k=1}^{K-2} \theta_k \, d_k(x) \tag{14}$$

où les fonction de base $d_k(x)$ pour k=1,...,K-2 ont pour expression:

$$d_k(x) = (x - \kappa_k)_+^3 - \frac{\kappa_K - \kappa_k}{\kappa_K - \kappa_{K-1}} (x - \kappa_{K-1})_+^3 + \frac{\kappa_{K-1} - \kappa_k}{\kappa_K - \kappa_{K-1}} (x - \kappa_K)_+^3. \tag{15}$$

Notez qu'il n'y a plus de contraintes sous cette forme!

L'ensemble des fonctions génératrices s'écrit donc

$$\{A, x, d_1(x), ..., d_{K-2}(x)\} \tag{16}$$

Il s'agit également d'une partie libre et donc d'une base de dimension K. Il y a K paramètres et 0 contraintes donc le nombre de degrés de liberté est de K.

L'avantage des splines cubiques naturelles c'est qu'elles sont très stables en extrapolation. En contrepartie, elles sont moins flexibles aux extrémités. De façon assez logique, les splines cubiques naturelles sont très utilisées quand les données sont limitées sur les bords.

Néanmoins, les incovénients qu'étaient la sensibilité au nombre de noeuds choisis et à la position de ces noeuds demeurent.

3 Splines cubiques pénalisées

On garde une base riche de K vecteurs de base mais on pénalise les fonctions trop "tordues".

3.1 Problème de minimisation

On cherche une fonction f qui minimise

$$\underbrace{\sum_{i=1}^{n}(y_i - f(x_i))^2}_{\text{Ajustement}} + \overbrace{\lambda \int [f''(t)]^2 d}^{\text{Rugosité}}, \tag{17}$$

qui est compromis entre ajustement aux données (goodness of fit) et pénalisation de la rugosité de la fonction via la courbure. Autrement dit, on cherche une fonction qui colle aux données tout en étant le plus lisse possible.

3.2 Décomposition de f en vecteurs de base

Repartons de l'expression des splines naturelles cubiques

$$f(x) = \beta_0 + \beta_1 x + \sum_{k=1}^{K-2} \theta_k \, d_k(x). \tag{18}$$

En changeant les notations dans une perspective d'écriture plus compacte (les termes β_0 et β_1x sont inclus dans la sommation), 18 peut se réécrire

$$f(x) = \sum_{k=1}^{K} \beta_k \phi_k(x) \tag{19}$$

avec commme base uniforme de dimension K

$$\{\phi_1(x) = 1, \phi_2(x) = x, \phi_3(x) = d_1(x), ..., \phi_K(x) = d_{K-2}(x)\}$$

Toutes les fonctions ϕ_k ne sont pas traitées de la même façon. Il convient de distinguer

- L'espace non pénalisé (car $f''(x) = 0$) engendré par les fonctions de base

  $$\phi_1(x) = 1, \phi_2(x) = x,$$

  qui est de dimension 2.

- L'espace pénalisé engendré par les fonctions de base

  $$\phi_3(x) = d_1(x), ..., \phi_K(x) = d_{K-2}(x),$$

  qui est de dimension K-2.

Remarque: Il n'est pas obligatoire de partir d'une spline cubique naturelle pour définir une spline pénalisée. On aurait très bien pu partir d'une spline cubique et l'exprimer comme 19 en utilisant une base de dimension K+4

$$\{\phi_1(x) = 1, \phi_2(x) = x, \phi_3(x) = x^2, \phi_4(x) = x^3$$
$$, \phi_5(x) = (x - \kappa_1)_+^3, ..., \phi_{K+4}(x) = (x - \kappa_K)_+^3\}$$

Ici aussi l'espace non pénalisé aurait été identique et de dimension 2 et l'espace non pénalisé de dimension K+2.

Alors pourquoi utiliser les splines naturelles ? Parce qu'elles ont déjà un bon comportement aux bords.

3.3 Passage à la forme matricielle

a) <u>Terme d'ajustement</u>

On souhaite écrire sous forme matricielle le terme d'ajustement

$$\sum_{i=1}^{n} (y_i - f(x_i))^2.$$

On observe les données $(x_i, y_i), i = 1, ..., n$

$$\begin{pmatrix} f(x_i) \\ f(x_2) \\ \vdots \\ f(x_n) \end{pmatrix} = \begin{pmatrix} \sum_{k=1}^{K} \beta_k \phi_k(x_1) \\ \sum_{k=1}^{K} \beta_k \phi_k(x_2) \\ \vdots \\ \sum_{k=1}^{K} \beta_k \phi_k(x_n) \end{pmatrix} = \underbrace{\begin{pmatrix} \phi_1(x_1) & \phi_2(x_1) & \dots & \phi_K(x_1) \\ \phi_1(x_2) & \phi_2(x_2) & \dots & \phi_K(x_2) \\ \vdots & \vdots & \vdots & \vdots \\ \phi_1(x_n) & \phi_2(x_n) & \dots & \phi_K(x_n) \end{pmatrix}}_{X} \underbrace{\begin{pmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_K \end{pmatrix}}_{\beta}$$

Donc:

$$f(x_i) = (X\beta)_i \tag{20}$$

et

$$\sum_{i=1}^{n} (y_i - f(x_i))^2 = \sum_{i=1}^{n} (y_i - (X\beta)_i)^2 = \|y - X\beta\|^2 \tag{21}$$

b) <u>Terme de pénalité</u>

On souhaite écrire sous forme matricielle le terme de pénalité

$$\lambda \int [f''(t)]^2 dt.$$

Etant donné,

$$f(x) = \sum_{k=1}^{K} \beta_k \phi_k(x), \tag{22}$$

on a :

$$f''(x) = \sum_{k=1}^{K} \beta_k \phi_k''(x). \tag{23}$$

On peut donc écrire

$$\begin{aligned}
\int (f''(t))^2\, dt &= \int \left(\sum_{i=1}^{K} \beta_i \phi_i''(t) \right) \left(\sum_{j=1}^{K} \beta_j \phi_j''(t) \right) dt \\
\Leftrightarrow \int (f''(t))^2\, dt &= \sum_{i=1} \sum_{j=1} \beta_i\, \beta_j \int \phi_i''(t) \phi_j''(t)\, dt \\
\Leftrightarrow \int (f''(t))^2\, dt &= \sum_{i=1} \sum_{j=1} \beta_i\, \beta_j S_{ij} \\
\Leftrightarrow \int (f''(t))^2\, dt &= \beta^T S \beta \qquad (24)
\end{aligned}$$

En vertu de (21) et (24), le problème (17) peut être réécrit sous la forme matricielle:

$$\min_{\beta} \|y - X\beta\|^2 + \lambda \beta^T S \beta \tag{25}$$

où pour résumer

- la matrice X encode les données, chacune de ces colonnes correspondant à une fonction de base.
- la matrice S, symétrique et semi-définie positive, encode la courbure. Elle dépend uniquement de la base choisie et non des données.

il est à noter que e problème est exactement une régression de ridge généralisée.

La solution du problème (25) est

$$\hat{\beta} = (X^T X + \lambda S) X^T y$$

où le chapeau rappelle qu'il s'agit d'un estimateur du vrai β qui dépend des données y (si on change les données, on change $\hat{\beta}$).

3.4 Rôle de λ

Le coefficient λ contôle le compromis.

λ	**Effet**
0	spline libre (overfit)
petit	flexible
grand	lisse
$+\infty$	droite

λ n'est pas fixé à la main. Il est estimé à partir des données de sorte d'optimiser la capacité prédictive du modèle tout en contrôlant sa complexité. On peut le voir comme un espèce de bouton de lissage.

3.4.1 Cross Validation (CV)

Étape 1 : solution du modèle

Pour un λ donné,

$$\hat{\beta}^{\lambda} = (X^T X + \lambda S)^{-1} X^T y$$

et donc

$$\hat{y} = X\hat{\beta}^{\lambda} = X(X^T X + \lambda S)^{-1} X^T y = A_{\lambda} y$$

Etape 2: leave-one-out CV

On calcule:

$$CV(\lambda) = \sum_{i=1}^{n} (y_i - \hat{y}_{-i}(x_i))^2$$

où $\hat{y}_{-i}$ est la prédiction sans le point i.

Le problème est que recalculer n modèles est coûteux en temps de calcul.

On peut démontrer que

$$\hat{y}_{-i}(x_i) = \frac{\hat{y}_i - A_{ii} y_i}{1 - A_{ii}}$$

ce qui permet de réécrire l'erreur de cross-validation par

$$\begin{aligned}
CV(\lambda) &= \sum_{i=1}^{n} (y_i - \hat{y}_{-i}(x_i))^2 \\
\Leftrightarrow CV(\lambda) &= \sum_{i=1}^{n} \left(y_i - \frac{\hat{y}_i - A_{ii} y_i}{1 - A_{ii}} \right)^2 \\
\Leftrightarrow CV(\lambda) &= \sum_{i=1}^{n} \left(\frac{y_i - y_i A_{ii} - \hat{y}_i - A_{ii} y_i}{1 - A_{ii}} \right)^2 \\
\Leftrightarrow CV(\lambda) &= \sum_{i=1}^{n} \left(\frac{y_i - \hat{y}_i}{1 - A_{ii}} \right)^2
\end{aligned}$$

Cette deuxième version de la formule permet effectivement d'éviter de devoir recalculer n modèles mais cela reste coûteux et parfois instable de calculer la diagonale de la matrice de lissage $(A_{\lambda})_{ii}$.

Pour déterminer $\hat{\lambda}$, on cherche

$$\hat{\lambda} = \arg\min_{\lambda} CV(\lambda)$$

soit par grid search (dans le cas où on teste des valeurs discrètes de λ) soit par une méthode de Newton (dans le cas où on un intervalle continue de λ). Généralement grid search est préférée à la méthode de Newton car $CV(\lambda)$ est

généralement irrégulière et parfois bruitée ce qui implique que les méthodes à dérivée peuvent être instables. En théorie, on peut donc utiliser grid search ou Newton pour CV, mais en pratique on utilise presque toujours grid search, car la fonction CV est trop coûteuse et peu régulière pour des méthodes basées sur les dérivées.

Concrètement grid search signifie simplement que l'on choisit une grille de λ et pour chaque valeur de λ, on calcule $CV(\lambda)$. On choisit alors la valeur de λ qui donne la $CV(\lambda)$ la plus faible. Les inconvénients de cette méthode est que c'est lent que la précision est limitée par la grille de λ choisie.

3.4.2 Generalized Cross-Validation)

L'idée est de remplacer les $(A_\lambda)_{ii}$ par une moyenne globale.

$$GCV(\lambda) = \frac{\|y - \hat{y}\|}{(n - trace(A_\lambda)^2}$$

On a

$$\text{edf} = \text{trace}(A_\lambda) \tag{26}$$

Intuitivement, le numérateur symbolise l' erreur d'ajustement et le dénominateur pénalise la complexité.

Pour déterminer $\hat{\lambda}$, on cherche

$$\hat{\lambda} = \arg\min_{\lambda} GCV(\lambda)$$

soit par grid search (dans le cas où on teste des valeurs discrètes de λ) soit par une méthode de Newton (dans le cas où on teste des valeurs continues de λ). Cette méthode consiste à chercher un minimum de la fonction $GCV(\lambda)$ en suivant la pente de la fonction.

3.4.3 REML = Restricted Maximum Likelihood (méthode moderne)

Il s'agit de la méthode standard (ex: mgcv). La REML (Restricted Maximum Likelihood) est une méthode pour choisir le paramètre de lissage λ en reformulant la spline pénalisée comme un modèle probabiliste : les coefficients associés à la courbure sont vus comme des effets aléatoires gaussiens, dont la variance est contrôlée par λ. Plutôt que de maximiser la vraisemblance des données brutes (comme en ML), la REML maximise la vraisemblance des résidus, c'est-à-dire la partie des données indépendante des effet fixes, c'est-à-dire des effets non pénalisés (comme 1 et x). Cela permet d'obtenir une estimation plus fiable de λ en corrigeant le biais du ML et en équilibrant automatiquement qualité d'ajustement et complexité du modèle.

3.5 Philosophie des splines pénalisées vs pilosophie des splines cubiques

Alors que dans le splines de régression le nombre de noeuds choisis contrôlait la complexité, ce n'est plus le cas pour les splines pénalisées car, dans ce cadre, c'est le choix de λ qui gouverne la complexité du modèle. Plus précisément, dans les splines pénalisées, le nombre de noeuds fixe la complexité maximale, et λ (via REML/GCV) contrôle la complexité réelle.

L'idée est de choisir beaucoup de noeuds pour construire une base flexible et d'ensuite laisser λ faire son travail de pénalisation. En pratique, tant que le nombre de nœuds est suffisamment grand, le résultat final dépend très peu du choix exact du nombre des noeuds et le résultat sera essentiellement gouverné par λ. En résumé, le nombre suffisamment grand de nœuds sert à définir une base riche, mais c'est la pénalisation (via λ) qui contrôle réellement la complexité du modèle.

Certes le nombre de noeuds ne fixe pas directement la complexité mais il fixe tout de même la complexité maximale possible. Intuitivement, on peut le voir comme λ qui fixe la complexité effective qui sera forcément inférieur au plafond fixé par le nombre total de noeuds K. Si on choisit un K trop petit, le modèle ne sera pas assez flexible et les résultats seront fortement biaisés. Si on choisit un K trop grand c'est moins grave puisque le modèle restera contrôle par λ. Néanmoins, cela augmentera le temps de calcul et les potentielles instabilités numériques. En pratique (ex: R mgcv), par défaut K=10 et on augmente si nécessaire. **Test important pour vérifier si la valeur de K choisie est suffisante**

Regarder les edf après estimation :

- si $edf < k - 1$, $\rightarrow$ la valeur de K est ok
- si $edf \approx k - 1$, $\rightarrow$ il faut augmenter la valeur de K

Notons que l'emplacement des noeuds est choisi par défaut pour avoir des intervalles réguliers.

3.6 Théorème des splines de lissage

La solution du problème de minimisation est une spline cubique naturelle avec des noeuds exactement sur chaque point de donné $x_1, x_2, ...x_n$. Cette splien est appellée spline de lissage.

Le théorème des splines de lissage établit que la pénalisation de la courbure conduit automatiquement à une spline cubique naturelle. En d'autres termes, on part d'un problème infini-dimensionnel (toutes les fonctions possibles) et la solution appartient automatiquement à l'espace des splines cubiques naturelles.

En pratique, on ne prend pas n nœuds car ce serait trop coûteux. On prend $K \ll n$ et on approxime donc avec une spline pénalisée (base K) la solution théorique optimale qui est la spline de lissage.

En résumé, ce théorème dit que les splines pénalisées de lissage (de type smoothing spline) sont exactement des splines cubiques naturelles, et que les splines pénalisées pratiques en sont des approximations.

Les splines pénalisées ne sont pas forcément des splines naturelles ($f''(x) = 0$ aux bords) car rien ne l'impose explicitement. Néanmoins, la pénalité pousse vers peu de courbure donc les splines pénalisées sont en pratique souvent presque linéaire aux bords. Mais ce n'est pas garanti stictement.

4 Thin plate spline

4.1 Origine du nom

Le nom vient d'une analogie physique très importante : thin plate = plaque métallique fine.

Imagine une fine plaque de métal (comme une tôle très mince) où l'on fixe quelques points dans l'espace. La plaque se déforme donc pour passer par ces points fixes tout en minimisant son énergie de flexion. Plus la surface est "pliée", plus elle coûte de l'énergie et est donc pénalisée. Donc la solution est la surface la plus lisse possible qui passe près des données.

4.2 Problème variationnel

On cherche une fonction f qui minimise

$$\underbrace{\sum_{i=1}^{n}(y_i - f(x_i))^2}_{\text{Ajustement}} + \overbrace{\lambda J_m(f)}^{\text{Rugosité}}, \tag{27}$$

où

$$J_m(f) = \int_{R^d} \sum_{|\alpha|=m} \binom{m}{\alpha} (D^\alpha f(x))^2 \, dx \tag{28}$$

avec les notations

- $\alpha = (\alpha_1, ..., \alpha_d)$: multi indice
- $|\alpha| = \alpha_1 + ... + \alpha_d = m$
- $D^\alpha f = \dfrac{\partial^m f}{\partial x_1^{\alpha_1} ... \partial x_d^{\alpha_d}}$

- $\binom{m}{\alpha} = \frac{m!}{\alpha_1!...\alpha_d!}$

Le problème de minimisation (27)peut être interprété comme un compromis entre ajustement aux données (goodness of fit) et pénalisation de la rugosité de la fonction via la courbure globale. Autrement dit, on cherche une fonction qui colle aux données tout en étant le plus lisse possible.

Intuitivement, (28) peut s'interpéter comme l'intégrale d'une somme des carrés de toutes les dérivées partielles d'ordre total m avec des poids combinatoires.

- Quand $d = 1$ et $m = 2$, la pénalité (28) devient

$$J_2(f) = \lambda \int [f''(t)]^2 \, dt, \tag{29}$$

  ce qui ramène exactement au problème de minimisation des splines pénalisées. Comme on l'a vu, dans ce cas, la solution du problème de minismisation est une spline cubique naturelle avec nœuds aux x_i.

- Quand $d = 2$ et $m = 2$, la pénalité (28) devient

$$J_2(f) = \lambda \int \left[\left(\frac{\partial^2 f}{\partial x^2} \right)^2 + 2 \left(\frac{\partial^2 f}{\partial x \partial y} \right)^2 + \left(\frac{\partial^2 f}{\partial y^2} \right)^2 \right] dt. \tag{30}$$

4.3 Solution exacte : thin plate spline

Le théorème fondamental (Wahba) nous dit que le f minimiseur du problème de minimisation (27) existe, est unique, et s'écrit sous la forme:

$$f(x) = \underbrace{\sum_{i=1}^{n} \alpha_i \psi(r)}_{\text{partie radiale}} + \underbrace{\sum_{j=1}^{M} \beta_j p_j(x)}_{\text{partie polynomiale}} \tag{31}$$

où

- $$\psi(r) = \begin{cases} r^{2m-d} \log(r) & \text{si m est pair} \\ r^{2m-d} & \text{si m est impair} \end{cases} \quad \text{avec } r = \|x - x_i\| \, .$$

- les $p_j(x)$ sont tous les monômes possibles en dimension d jusqu'au degré m-1. En tout il y en a

$$M = \binom{d + m - 1}{m - 1}.$$

  - Dans le cas classique pour $d = 1$ et $m = 2$,

$$M = \binom{1 + 2 - 1}{2 - 1} = \binom{2}{1} = \frac{2!}{1!1!} = 2,$$

la partie polynomiale se réécrit simplement

$$\beta_0 p_1(x) + \beta_1 p_2(x) = \beta_0 1 + \beta_1 x.$$

– Dans le cas où $d = 2$ et $m = 2$,

$$M = \binom{2+2-1}{2-1} = \binom{3}{1} = \frac{3!}{1!2!} = 3,$$

la partie polynomiale se réécrit simplement

$$\beta_0 p_1(x) + \beta_1 p_2(x) + \beta_2 p_3(x) = \beta_0 1 + \beta_1 x_1 + \beta_2 x_2.$$

.

– Dans le cas où $d = 3$ et $m = 2$,

$$M = \binom{3+2-1}{2-1} = \binom{4}{1} = \frac{4!}{1!3!} = 4,$$

la partie polynomiale se réécrit simplement

$$\beta_0 p_1(x) + \beta_1 p_2(x) + \beta_2 p_3(x) + \beta_3 p_4(x) = \beta_0 1 + \beta_1 x_1 + \beta_2 x_2 + \beta_2 x_3.$$

- il y a M contraintes qui assurent unicité de la solution et séparation entre partie polynomiale et radiale:

$$\sum_{i=1}^{n} \alpha_i p_j(x_i) = 0 \text{ avec } j = 1, ..., M$$

– Dans le cas classique où $d = 1$ et $m = 2$, c'est-à-dire $M = 2$, les contraintes s'écrivent:

$$\begin{cases} \text{constante} & \sum_{i=1}^{n} \alpha_i = 0 \\ \text{direction } x & \sum_{i=1}^{n} \alpha_i x_i = 0 \end{cases}$$

– Dans le cas où $d = 2$ et $m = 2$, c'est-à-dire $M = 3$, les contraintes s'écrivent:

$$\begin{cases} \text{constante} & \sum_{i=1}^{n} \alpha_i = 0 \\ \text{direction } x_1 & \sum_{i=1}^{n} \alpha_i x_{i1} = 0 \\ \text{direction } x_2 & \sum_{i=1}^{n} \alpha_i x_{i2} = 0 \end{cases}$$

Les contraintes imposent que la partie radiale soit orthogonale aux polynômes de vase de la partie polynomiale, garantissant une séparation propre et unique entre tendance globale (partie polynomiale) et variations locales (partie radiale). Sans ces contraintes, il y aurait ambiguïté entre ce qui est capturé par les α_i et ce qui est capturé par les β_j.

Interprétons maintenant l'expression d'une thin plate spline (31) en dimension générale. Elle s'écrit comme la somme de deux composantes :

- une partie radiale, constituée d'une combinaison de n fonctions centrées sur les points d'observation, et
- une partie polynomiale, de degré inférieur ou égal à m-1, qui engendre l'espace nul de la pénalité.

La partie radiale décrit les variations locales de la fonction, tandis que la partie polynomiale, non pénalisée, capture la tendance globale. Les contraintes d'orthogonalité assurent une décomposition unique en évitant toute redondance entre ces deux composantes.

La solution exacte pour $d = 1$ et $m = 2$:

$$f(x) = \sum_{i=1}^{n} \alpha_i \left|x - x_i\right|^3 + \beta_0 + \beta_1 x.$$

On a donc une base de taille n, chaque point de donnée étant un centre d'une fonction de base. La solution de ce problème est donc une spline cubique naturelle avec nœuds aux x_i. Ce résultat était attendu puisqu'on a vu plus haut que pour d=1 et m=2, le problème de minimisation (27) était identique au problème de minimisation des splines pénalisées. Or on a vu que le théorème des splines de lissage nous assurait que la la solution du problème de minisation était une spline cubique naturelle avec des noeuds sur chaque point de donnée.

Rem: Il y a une différence apparente entre entre spline naturelle (de base $(x - x_i)_+^3$) et thin plate spline (de base $|x - x_i|^3$) mais en réalité ces deux bases engendrent le même espace fonctionnel étant donné qu'une fonction radiale s'écrit comme combinaison de deux fonctions tronquées : $|x - x_i|^3 = (x - x_i)_+^3 + (x_i - x)_+^3$. Autrement dit, toute fonction f écrite avec $|x - x_i|^3$ peut être réécrite avec $(x - x_i)_+^3$ et inversément.

4.4 Réduction de dimension

La solution exacte utilise n fonctions de base $\phi(\|x - x_i\|)$, chaque point de donnée étant un centre d'une fonction de base. Le problème est que cela rend le temps de calcul très long pour des n grands.

L'idée clé de Wood fut d'approximer la solution avec une base de dimension réduite $k \ll n$.

$$f(x) = \sum_{k=1}^{K} \beta_k \phi_k(x), \tag{32}$$

avec K petit (typiquement entre 10 et 50) et $\phi_k(x)$ des nouvelles fonctions de base qui sont consruites ingénieusement.

La réduction de dimension dans les thin plate splines consiste à partir de la solution théorique exacte - qui est une combinaison de n fonctions radiales centrées sur chaque point de données - et à la projeter dans un sous-espace beaucoup plus petit mais bien choisi. Concrètement, on commence par construire la matrice noyau K, dont les éléments mesurent l'influence entre les points via la fonction radiale $\phi(\|x - x_i\|)$. Cette matrice encode toute la structure de lissage du problème. On effectue ensuite une décomposition spectrale de K, ce qui permet d'identifier des directions principales (vecteurs propres) associées à des niveaux décroissants de "variation lisse" (valeurs propres). L'idée clé est que les fonctions les plus pertinentes pour représenter une fonction régulière correspondent aux plus grandes valeurs propres ; on ne conserve donc que les K premières composantes (avec $K \ll n$). Ces composantes définissent une nouvelle base de fonctions ϕ_k, chacune étant une combinaison des fonctions radiales initiales. On réécrit alors le problème dans cette base réduite, ce qui donne une spline pénalisée classique de dimension K, avec une pénalité adaptée. Cette projection permet de conserver l'essentiel de la structure lisse de la solution exacte tout en réduisant drastiquement le coût de calcul, car on passe d'un problème de taille n à un problème de taille K, sans perdre les propriétés fondamentales de régularisation.

5 Package mgcv de R

5.1 Fonction gam

$$gam(y\ s(x, bs = "...", k = ...), method = data = ...)$$

s(x) construit une fonction lisse f(x).

5.1.1 Choix du type de spline : bs=?

- bs="tp" → thin plate spline.
  C'est le choix par défaut. Correspond directement à ce qu'on a vu (Wahba).
  - Le nombre de noeuds K qui fixe la complexité maximale est par défaut de 10. Il faudra vérifier si ce nombre K était suffisamment grand. Si ce n'est pas le cas, il faudra l'augmenter.
  - Base construite automatiquement (troncature).
  - Très stable, peu de tuning
- bs="ps" → P-spline → très robuste
- bs="cr" → cubic regression spline → plus simple
- bs="cs" → cubic shrinkage spline → version pénalisée

Quand on ne sait pas quoi choisir, il est consseilé d'opter pour les thin plate splines (choix par défaut) ou les P-splines car cela évite de devoir réfléchir aux noeuds et de devoir optimiser manuellement.

5.1.2 Choix de la dimension de la base: K=?

Par défaut,

- pour des fonctions d'une variable, K=10.
- pour des fonctions de 2 variables:

Type	Syntaxe	k par défaut	Interprétation
Isotrope	`s(x, y)`	30	Complexité globale
Tensoriel	`te(x, y)`	$c(5, 5)$	Complexité par dimension

Table 1: Différence entre splines bidimensionnelles dans mgcv

La valeur de k fixe la complexité maximale. Il faudra vérifier avec les outputs du modèle, si ce nombre K était suffisamment grand. Si ce n'est pas le cas, il faudra l'augmenter et relancer la modélisation.

- Pour "tp", k donne la dimension de la base tronquée (approximation des thin plate splines).

- Pour "cr", k donne la dimension de la base spline et détermine implicitement un ensemble d'environ k nœuds (placés aux quantiles), qui structurent la base.

5.1.3 Choix de la méthode d'estimation du paramètre de pénalisation: λ=?

Comme expliqué plus haut, choisisesse la méthode "REML" qui est la méthode par défaut.

6 Création d'un zonier

6.1 Introduction

Un zonier consiste à découper un territoire en zones tarifaires selon le risque. L'idée va être de construire à l'aide d'un modèle gam, une surface de risque lisse en fonction de la localisation, puis de la discrétiser en zones.

Question: Faut-il ou non tenir compte des variables non géographques dans la construction du zonier?
Si on inclut pas les autres variables explicatives (âge, puissance,..etc), le spatial va absorber à la fois les effets géographiques réels mais également les effets liés aux autres variables (ex: quartiers jeunes) et il en résultera un zonier biaisé.

En conclusion, même si cela peut paraître paradoxal, il faut inclure les autres variables pour isoler le “vrai effet spatial”.

Question: Peut-on faire un zonier sur les résidus d’un premier modèle qui capterait tous les effets non géographiques?
Non, car les résidus d’un premier modèle ne constituent pas une variable réponse appropriée pour construire un zonier. En effet, ces résidus sont centrés (espérance nulle) et ne suivent pas une distribution de la famille exponentielle, ce qui viole les hypothèses fondamentales des modèles utilisés pour la tarification. De plus, ils dépendent du modèle initial et de son éventuelle mauvaise spécification : toute structure non captée (effets non linéaires, interactions, hétérogénéité) peut se retrouver dans les résidus et être à tort attribuée à un effet géographique. Ainsi, un zonier construit sur les résidus risquerait de capter des artefacts du modèle plutôt qu’un véritable signal spatial, conduisant à des biais d’interprétation et de tarification.

Question: Peut-on faire un zonier en mettant en offset un premier modèle qui capterait tous les effets non géographiques?
Oui, c’est une option possible de construire un zonier en introduisant en offset un premier modèle capturant les effets non géographiques, car cela revient à décomposer l’intensité en une composante déjà estimée et une composante spatiale à expliquer. Cependant, cette approche n’est généralement pas recommandée, car elle repose implicitement sur une hypothèse forte d’indépendance entre les effets géographiques et non géographiques. En pratique, ces effets peuvent être corrélés (par exemple, certaines caractéristiques socio-démographiques ou comportementales variant selon la zone), de sorte que figer la partie non spatiale via un offset empêche de réajuster conjointement les paramètres et peut conduire à des biais d’estimation ou à une mauvaise attribution des effets. Une modélisation conjointe est donc préférable afin de tenir compte des interactions potentielles et de l’incertitude globale.

Etape 1: Modélisation

```r
model <- gam(N ~ s(lon, lat, bs="tp",k=100)
                 + Vehgas + s(age) + offset(log(exposure)),
                 method="REML",
                 family = poisson,
                 data = df)
```

- *Choix du type de spline*

  - bs="tp"=on choisit une thin plate splines 2D: isotrope, pas de noeuds à choisir, très adatpé spatial
  - bs="te": on choisit un tensor product spline car elle gère mieux les échelles différentes (latitude vs longitude) et l'anisotroie. Alternative souvent meilleure en pratique.

- *Choix du nombre de fonctions de base K*
  K est le nombre de fonctions de base 2D qui fixe la complexité maximale du modèle. Il convient en de commencer avec k=50 à 200 et de vérifier si c'est suffisant avec la fonction gam.check(model)

- *Choix de la méthode d'estimation du paramètre λ: REML*

Etape 2: Validation

```
gam.check(model)
```

Si on a en même temps:

- edf $\approx k'$ où k' est la dimension effective maximale du terme lisse après contraintes d'identifiabilité (généralement k'=K-1).

- $k_{index} < 1$ où k-index

- p-valeur faible (l'hypothèse nulle étant k est suffisant, un p-value inférieur à 5% signifie que l'on rejette l'hypothèse nulle ce qui signifie que K est insuffisant.

cela signifiqe qu'il faut augmenter K.

Etape 3: Construction de la surface de risque

Il existe trois stratégies pour la construction de la surface de rique (pas encore des zones proprement dite).

- Option 1: consiste à extraire directement le terme spatial estimé, $f_{spatial}(lat, long)$, via predict(..., type = "terms"). On obtient alors une quantité centrée (identifiable à une constante près), telle que $\sum_{i=1}^{n} f_{spatial}(lat_i, long_i)$, et donc seulement un effet relatif :

  $$\hat{N}(lat, long) \propto exp(f_{spatial}(lat, long))$$

  Cette représentation est suffisante pour analyser la structure spatiale ou construire des zones (clustering), mais elle ne fournit pas directement un

niveau de risque exploitable sans recalibration. Ainsi, l'approche conditionnelle, qui est l'option 2) est privilégiée pour la tarification et l'interprétation, tandis que l'extraction du terme spatial est une alternative pratique pour l'analyse relative et la segmentation.

- Option 2 : zonier conditionnel ("toutes choses égales par ailleurs") [recommandée]

  On fixe les variables non spatiales (en prenant la moyenne pour les variables continues et la modalité la plus peuplée pour les variables catégorielles). L'objectif est de capturer l'effet de la localisation à profil constant. Le zonier répondre à la question: "À profil égal, comment varie le risque selon la localisation ?" Si on note N(lat,long) le modèle gam qui prédit le nombre de sinistres, cela revient à définir

  $$\hat{N}(lat, long) = \exp(f_{spatial}(lat, long) + \text{ reste des variables fixées })$$

  Est-ce que le choix du profil fixé a de l'importance? Le terme dépendant du profil fixé est donc une constante multiplicative. Changer le profil multiplie toute la carte par une constante mais ne change pas la forme de la surface de risque. On en conclut que le classement des zones ne dépend pas de du profil fixé. Par contre, le niveau de risque dépend du profil fixé. Autrement dit, si l'objectif est uniquement le zonier (segmentation), alors le choix du profil fixé n'a pas d'importance. Si l'objectif est de faire de la tarification, le choix du profil fixé devient important. A choisir, on préféra donc fixer un profil puisque cela permet d'avoir un niveau interprétable. Mais dans cas, quel profil choisir? Intuitivement, il paraît faire sens puisqu'on cherche à interpréter le niveau, à choisir un espèce de profil moyen et d'opter pour

  - la moyenne pour les variables continues. Ce n'est pas une obligation mais une convention raisonnable.
  - la catégorie de référence pour les variables catégorielles. On pourrait aussi choisir la modalité la plus peuplée ou une modalité "standard métier".

  Le zonier qui en résultera capturera l'effet spatial pur. C'est l'option recommandée et l'option standard en actuariat.

- Option 3 : zonier marginal (réaliste portefeuille)

  Pour chaque couple (long, lat), on moyenne sur les autres variables. Autrement dit, on prend la moyenne du risque tel qu'il est réellement dans cette zone. Mathématiquement, cela revient pour chaque couple (lat,long) à moyenner toutes les prédictions des individus du portefeuille qui habitent à (lat,long) ou proche. Si on note n_s ce nombre d'individus, on a:

  $$\hat{N}(lat, long) \approx \frac{1}{n_s} \sum_{i=1}^{n_s} \exp(f_{spatial}(lat_i, long_i) + \text{reste}_i)$$

Le zonier marginal est la moyenne empirique pondérée par l'exposition des prédictions du GAM sur les individus présents dans chaque zone. La surface de risque ainsi construite reflètera le portefeuille tel qu'il est distribué spatialement mais mélangera des effets gégographiques et les effets des autres variables.
Signalons que si les zones existent déjà, on effecteura la moyenne des prédictions sur tous les individus habitant dans cette zone. *Exemple permettant de comprendre la différence entre les deux options* Soient:

- Zone A (centre-ville): jeunes conducteurs petites voitures plus de sinistres
- Zone B (campagne) conducteurs âgés moins de sinistres

Le zonier conditionnel compare A et B avec même profil et mesure donc le danger intrinsèque de la zone. Le zonier marginal compare A et B avec leurs populations réelles puisqu'il fait dans chaque zone la moyenne de la variable target (nombre de sinitres) de la population de la zone en question. Dans notre exemple, les deux zones ont des populations très différentes et le zonier marginal construit mélangera donc les effets géographiques avec d'autres effets (ici âge).

La conclusion est assez claire. **Il est préférable d'opter pour le zonier conditionnel avec profil fixé!**

En pratique, cela signifie que l'on effectue les prédictions sur une grille dans laquelle les variables non spatiales sont fixées.

$$\hat{N}(lat, long, Age, Vehgas) = \exp\left(\beta_0 + f_{spatial}(lat, long) + f_{Age}(Age) + \beta_1 x_1\right) \tag{33}$$

On fixe un profil:

$$Age = \overline{Age} \text{ et } VehGas = Diesel \text{ (catégorie de référence).}$$

Cela implique que (33) se réécrit:

$$\hat{N}(lat, long) = \exp\left(\beta_0 + f_{spatial}(lat, long) + f_{Age}(\overline{Age})\right)$$

On définit une grille et on calcule les prédictions:

```r
# On d finit la grille
grid <- expand.grid(lon = seq(min(df$lon), max(df$lon), length = 100),
                    lat = seq(min(df$lat), max(df$lat), length = 100))
grid$age     <- mean(df$age)
grid$Vehgas <- "Diesel"
grid$expo    <- 1

# On effectue les predictions sur la grille sur base du mod le
grid$pred <- predict(model, newdata = grid, type = "response")
```

Etape 4: Discrétisation

L'objectif est de passer de la surface de risque lisse $f_{spatial}(lat, long)$ à un zonier exploitable, stable et géographiquement cohérent. Ci-après, nous présentons différentes options pour cette discrétisation.

Méthode 1 — Expertise métier

On fixe manuellement des seuils basés sur :

- expertise métier;
- normes réglementaires et
- interprétation du GAM.

```r
# Definition des seuils experts
breaks <- c(-Inf, 0.2, 0.5, 0.8, Inf)

# Discretisation
grid$zone <- cut(grid$pred, breaks = breaks,
                 labels = c("faible", "moyen", "eleve", "tres eleve"),
                 right = FALSE)

# Visualisation
table(grid$zone)
```

Les avantages sont l'interprétabilité et la stabilité. Les inconvénients sont le caractère subjectif et lefait qu'en procédant ainsi, on ignore la distribution des données.

Méthode 2 — Quantiles

On découpe de façon à avoir le même nombre d'observations par classe. Si on choisit K classes, chaque classe contient $\approx \dfrac{n}{K}$ observations.

```r
# Nombre de classes
K <- 4

# Calcul des quantiles
breaks <- quantile(grid$pred, probs=seq(0,1,length.out=K+1),na.rm=TRUE)

# Discretisation
grid$zone <- cut(grid$pred, breaks=breaks, include.lowest=TRUE, labels=
    paste0("Q",1:K))

# Verification
table(grid$zone)
```

Les avantages c'est que c'est équilibré (utile pour cartographie) et robuste aux valeurs extrêmes Les inconvénients sont que les classes parfois peu homogènes et les coupures arbitraires visuellement

Méthode 3 — Intervalles égaux

On découpe l'intervalle de variation en segments de même largeur.

```r
# Nombre de classes
K <- 4

# Creation des bornes
breaks <- seq(min(grid$pred, na.rm = TRUE),
              max(grid$pred, na.rm = TRUE),
              length.out = K + 1)

# Discretisation
grid$zone <- cut(grid$pred,breaks = breaks,include.lowest=TRUE)

table(grid$zone)
```

Méthode 4 — K-means 1D

On regroupe les observations en minimisant la variance intra-classe. On cherche à minimiser :

$$\sum_{k=1}^{K} \sum_{(lat_i, long_i) \in C_k} \left(\hat{N}(lat_i, long_i) - \mu_k \right)^2$$

où C_k est un cluster k et μ_k est le centre du cluster.

On considère les données triées: $x_1 \leq x_2 \leq ... \leq x_n$. L'algorithme k-means affecte chaque point au centre le plus proche:

$$cluster(x_i) = arg \min_k |x_i - \mu_k| \tag{34}$$

Or, entre deux centres μ_k et μ_{k+1}, la frontière est

$$b = \frac{\mu_k + \mu_{k+1}}{2}. \tag{35}$$

Donc:

- tous les points à gauche de b → cluster k;
- tous les points à droite → cluster k+1.

Par conséquent, chaque cluster est un intervalle continu $C_k = [a_k, a_{k+1}]$.

En résumé, la méthode k-means n'impose pas la contiguïté par construction

mais en dimension 1, k-means produit des classes contiguës.

Par contre, en dimension > 1, là les clusters ne sont plus des intervalles mais des régions $\|\mathbf{x} - \mu_k\|$. On peut avoir des formes complexes, non connexes visuellement.

Même si en 1D la méthode des k-means donne des intervalles contigus, la méthode Jenks est meilleure car

- k-means minimise localement, Jenks globalement.
- k-means est sensible à l'initialisation (résultats différents selon set.seed())
- k-means donne des coupures moins naturelles car k-means équilibre auour des centres alors que jenks détecte les "trous" dans les données.

En résumé, la méthode k-means découpe par proximité des centre alors que Jenks découpe optimalement en segments. En 1 D, les deux donnent des intervalles contigus mais k-means donne une solution approchée et Jenks une solution optimale.

```r
set.seed(123)
# Resultats differents selon set.seed()
km <- kmeans(grid$pred, centers = 4)

# Option : ordonner les classes
centers <- km$centers
order_centers <- order(centers)

grid$zone <- factor(km$cluster,levels = order_centers,labels = paste0(
    "C", 1:4))

# Visualisation
table(grid$zone)
```

Méthode 5 - Jenks (Natural Breaks)

L'objectif est de trouver des classes internement homogènes et bien séparées entre elles. Autrement dit, le but est de minimiser la variance intra-classe et maximiser la variance inter-classe.

On considère les données triées: $x_1 \leq x_2 \leq ... \leq x_n$ et on cherche à construire K classes. le but est de trouver les indices

$$1 = i_0 < i_1 < i_2 < ... < i_K = n$$

tels que les classes soient:

$$C_k = \{x_{(i_{k-1}+1)}, ..., x_{(i_k)}\}.$$

Autrement dit, la contiguïté est imposée dans la définition des classes $C_k = \{x_{(j)} : j \in [i_{k-1}+1; i_k]\}$ et on ne cherche la solution "partition optimale" que parmi les partitions en segments contigus. Sans cette contrainte, on retombe exactement sur la méthode des k-means.

la fonction objectif que l'on cherche à minimiser est

$$SSW = \sum_{k=1}^{K} \sum_{j=i_{k-1}+1}^{i_k} \left(x_{(j)} - \mu_k\right)^2$$

avec

$$\mu_k = \frac{1}{i_k - i_{k-1}} \sum_{j=i_{k-1}+1}^{i_k} x_{(j)}$$

On ne va pas expliquer en détails l'agorithme qui va permettre de construire les classes k en minisant la fonction objectif mais la philosphie est que l'on teste tous les points de coupure possibles mais toujours en gardant des segments contigus et que l'on garde les classes qui minimisent SSW. La méthode de Jenks peut donc être vue comme la segmentation optimale d'une série triée. Contrairement au k-means 1D, la contiguîté n'est pas un résultat mais une contrainte stucturelle du problème. C'est souvent le standard en cartographie.

```r
library(classInt)

# Jenks
jenks <- classIntervals(grid$pred, n = 4, style = "jenks")

# Discretisation
grid$zone <- cut(grid$pred,breaks=jenks$brks,include.lowest = TRUE)

table(grid$zone)

# Visulaisation
hist(grid$pred, breaks = 30, col = "lightgray", main = "Jenks")

# ajouter les seuils
abline(v = jenks$brks, col = "red", lwd = 2)
```

Méthode 6 - Skater

Découpe un graphe en minimisant l'hétérogénéit

```r
library(spdep)

# matrice de voisinage
nb <- poly2nb(sf_data)

# ponderations
lw <- nb2listw(nb)

# distance sur variable
dist_mat <- dist(df$pred)

# arbre couvrant minimum
mst <- mstree(lw, dist_mat)

# decoupage en K groupes
groups <- cutree(mst, k = 4)

df$zone_skater <- factor(groups)
```

Méthode 7 - Clustering hiérarchique contraint (Ward spatial)

```r
 library(ClustGeo)

# distance attributaire
D0 <- dist(df$pred)

# distance spatiale (coordonnees)
coords <- cbind(df$lon, df$lat)
D1 <- dist(coords)

# compromis entre les deux
alpha <- 0.2

tree <- hclustgeo(D0, D1, alpha = alpha)

df$zone_ward <- cutree(tree, k = 4)
```